

IS PROFESSIONAL ELECTIVE #4
Generative AI Systems

SEMESTER PROJECT GUIDE

Mini Generative AI Capstone Project

1st Semester, A.Y. 2026–2027

Instructor: Jessie A. Melendres

1. Project Overview

This semester project is your hands-on Performance Task series for IS Professional Elective #4. Working individually or in a small team, you will design, build, and ship a working Generative AI application — your own mini capstone — that demonstrates everything covered in the course: text preprocessing and embeddings, prompt engineering, vector databases, Retrieval-Augmented Generation (RAG) orchestration, and deployment through containerized, production-style LLMOps practices.

Instead of one big project dropped on you at the end of the term, the work is broken into four checkpoints that line up with the four units of the syllabus and the four Performance Tasks already built into your grading scheme. Each checkpoint builds directly on the previous one, so by Finals you are not starting a project — you are completing one.

Checkpoints are exam-period anchored:

- Checkpoint 1 – Prelim (Weeks 1–4): Problem framing, data pipeline, embeddings
- Checkpoint 2 – Midterm (Weeks 5–8): Prompt architecture and vector indexing
- Checkpoint 3 – Semi-Final (Weeks 9–12): RAG orchestration and working application
- Checkpoint 4 – Final (Weeks 13–16): Fine-tuning rationale, deployment, and Final Presentation & Defense

2. Course Intended Learning Outcomes (CILOs) Addressed

This project is the primary vehicle for demonstrating all four CILOs of the course. Your final submission should give clear, working evidence of each:

CILO	What your project must demonstrate
CILO 1	A full end-to-end data pipeline: text cleaning/sanitation, tokenization, embedding generation, and (where relevant) dimensionality reduction.
CILO 2	A scalable RAG architecture using a vector database and an orchestration framework (LangChain and/or LlamaIndex).
CILO 3	Informed use of open-source foundation models, with at least an applied understanding of PEFT/LoRA/QLoRA fine-tuning concepts within memory constraints.
CILO 4	A secure, containerized, deployed application (e.g., a conversational bot or source-code analyzer) with a CI/CD-aware deployment workflow.

3. Choose Your Capstone Theme

Pick ONE problem you genuinely want to solve with a generative AI application. Your theme should be specific enough to scope a useful RAG knowledge base. Suggested directions (or propose your own, subject to instructor approval):

- Document/Knowledge-Base Q&A Assistant – e.g., a chatbot that answers questions over student handbooks, manuals, or research papers.
- Source Code Analyzer/Reviewer – ingests a codebase and answers questions, flags issues, or explains functions.
- Customer Support / FAQ Bot – grounded in a company's product docs or policies.

- Academic Research Assistant – summarizes and answers questions from a curated set of journal articles.
- Local Business / Community Assistant – e.g., tourism, agriculture, or health-advisory bot grounded in local data.
- Personal/Domain Notes Assistant – a RAG app over your own notes, recipes, legal texts, or similar structured/unstructured content.

Your theme must use a real (or realistically simulated) dataset of at least 20–30 text documents/pages so the RAG pipeline has something meaningful to retrieve from.

4. Team Formation & Conduct

- Work individually or in teams of 2–4 students, as announced by the instructor.
- Each teammate must own a clear pipeline component (e.g., data/embeddings, orchestration/RAG, interface/deployment) — division of labor must be declared in every checkpoint submission.
- All code must be version-controlled in a shared GitHub/GitLab repository from Checkpoint 1 onward. Commit history is part of your evidence of contribution.
- Academic integrity applies fully: cite any external code, datasets, or AI-assisted code generation you used, and be ready to explain any part of your own codebase during the defense.

5. Checkpoint Roadmap

The table below maps each checkpoint to its exam period, syllabus unit, and the Performance Task it fulfills.

Checkpoint	Weeks	Syllabus Unit Focus	Exam Period	Performance Task
Checkpoint 1	1–4	Foundational Pipeline & Linguistic Spaces	Prelim	Performance Task 1
Checkpoint 2	5–8	Commercial Engines, Engineering Prompts & Vector Indices	Midterm	Performance Task 2
Checkpoint 3	9–12	Orchestration, Knowledge Bases & Contextual Routing	Semi-Final	Performance Task 3
Checkpoint 4	13–16	Customization, Delivery & Operational Deployment	Final	Performance Task 4 + Final Defense

Checkpoint 1: Foundation & Data Pipeline

PRELIM • Weeks 1–4 • Project Weight: 20% of final Project grade

By the end of the Prelim period you must lock in your capstone theme and prove that you can take messy real-world text and turn it into something a model can reason over. This checkpoint is graded as Performance Task 1.

Required Deliverables

1. Project Proposal (1–2 pages): problem statement, target users, chosen theme, dataset source(s), and success criteria.
2. Data Collection: a raw dataset of at least 20–30 documents/pages relevant to your theme, stored in your repository (or linked if too large).
3. Text Preprocessing Notebook/Script: cleaning, normalization, and tokenization of the raw text, with before/after examples.
4. Embedding Generation Demo: code that converts a sample of your processed text into embeddings using a model of your choice (e.g., via Hugging Face or an API), with a short written explanation of the model you chose and why.
5. Short reflection (½ page): what was messy about your data and how you handled it.

Checkpoint Rubric

Criterion	Description	Points
Problem & Scope Clarity	Problem statement is specific, the dataset is appropriately scoped, and success criteria are measurable.	20
Data Preprocessing	Cleaning and tokenization are correctly implemented and clearly demonstrated with before/after evidence.	30
Embedding Generation	Embeddings are correctly generated and the model choice is justified.	30
Documentation & Repo Hygiene	Proposal is clear; repository is organized with a working README and commit history.	20
TOTAL		100

Checkpoint 2: Prompt Architecture & Vector Indexing

MIDTERM • Weeks 5–8 • Project Weight: 20% of final Project grade

This checkpoint proves you can design prompts that constrain model behavior and that you can stand up a queryable vector index over your embeddings. This checkpoint is graded as Performance Task 2.

Required Deliverables

6. Prompt Engineering Document: at least 3 structured system/instruction prompts designed for your use case, each with a stated purpose and at least one example showing how it prevents hallucination or off-topic answers.
7. API Integration Demo: a working script that sends your prompts to a Chat Completion API (e.g., OpenAI or an equivalent) and returns grounded responses.
8. Vector Database Setup: your embeddings loaded into a vector database (e.g., Chroma, FAISS, Pinecone, Weaviate) with a working similarity-search query.
9. Distance Metric Comparison: a short report (1 page) comparing Cosine, Euclidean, and Dot Product similarity on your own dataset, with example query results for each and a recommendation.

Checkpoint Rubric

Criterion	Description	Points
Prompt Design Quality	Prompts are structured, purposeful, and demonstrably reduce hallucination/off-scope answers.	25
API Integration	Chat Completion API calls work reliably and handle errors/edge cases sensibly.	20
Vector Database Implementation	Embeddings are correctly indexed and similarity search returns sensible, relevant results.	30
Distance Metric Analysis	Comparison is technically sound, with concrete examples and a defensible recommendation.	25
TOTAL		100

Checkpoint 3: RAG Orchestration & Working Application

SEMI-FINAL • Weeks 9–12 • Project Weight: 30% of final Project grade

This is the heart of the capstone: a real, runnable Retrieval-Augmented Generation application built on an orchestration framework. This checkpoint is graded as Performance Task 3.

Required Deliverables

10. Ingestion Pipeline: an automated pipeline that parses, chunks, embeds, and stores new documents end-to-end (not just the demo from Checkpoint 1).
11. RAG Application: a working application (CLI or simple UI) using LangChain and/or LlamaIndex that retrieves relevant context and grounds model responses in it.
12. Conversational Memory: session/state memory so the application can hold a multi-turn conversation, not just answer single isolated queries.
13. Framework Comparison Note (½–1 page): a short comparison of how LangChain and LlamaIndex each approach data routing/node indexing, and which parts of each you used and why.
14. Live demo (in-class or recorded) showing at least 5 realistic queries and grounded answers, including one query that should be correctly refused or flagged as out-of-scope.

Checkpoint Rubric

Criterion	Description	Points
Ingestion Pipeline	Pipeline reliably parses, chunks, embeds, and stores documents with minimal manual steps.	20
RAG Correctness	Retrieved context is relevant; answers are grounded in retrieved documents rather than hallucinated.	30
Orchestration & Memory	Framework is used appropriately; conversational memory works across multiple turns.	25
Demo & Evaluation	Demo queries are realistic and well-chosen; failure/out-of-scope case is handled gracefully.	15
Documentation	Framework comparison note is accurate and reflects actual implementation choices.	10
TOTAL		100

Checkpoint 4: Customization, Deployment & Final Defense

FINAL • Weeks 13–16 • Project Weight: 30% of final Project grade

The last stretch turns your working RAG prototype into something closer to a real product: containerized, deployable, and defensible in front of an audience. This checkpoint is graded as Performance Task 4 and includes your Final Presentation & Defense.

Required Deliverables

15. Model Customization Note: a short technical write-up (1–2 pages) on whether/how PEFT (LoRA/QLoRA) could improve your application, including any fine-tuning experiments attempted, or a well-reasoned justification for why you used the base/instruction-tuned model as-is.
16. Application Interface: your RAG logic wrapped behind a simple web interface (e.g., Streamlit, Gradio, or a minimal Flask/FastAPI front end).
17. Containerization: a working multi-stage Dockerfile that builds and runs your application cleanly, plus a short note on environment variables/secrets handling.
18. Deployment Evidence: proof of deployment (screenshots, a public/sandbox URL, or a recorded run) showing the containerized app running outside your local machine, ideally via a simple CI/CD step.
19. Final Technical Report (3–5 pages): consolidated documentation covering architecture, data pipeline, prompt design, RAG implementation, deployment, limitations, and lessons learned.
20. Final Presentation & Defense (see Section 7).

Checkpoint Rubric

Criterion	Description	Points
Model Customization Reasoning	PEFT/fine-tuning analysis is technically sound, whether or not fine-tuning was actually performed.	15
Application Interface	Interface is functional, usable, and correctly wraps the RAG logic.	15
Containerization	Dockerfile builds and runs without manual patching; image is reasonably clean/minimal.	20
Deployment Evidence	Application is demonstrably running in a containerized/deployed environment.	15
Final Report Quality	Report is complete, accurate, and clearly documents the full pipeline end-to-end.	15
Final Presentation & Defense	Team clearly explains and defends design decisions; answers panel questions accurately.	20
TOTAL		100

6. Final Presentation & Defense Guidelines

Scheduled during Finals week, alongside Performance Task 4. Each team presents and defends their capstone in front of the instructor (and panel, if assigned).

Format

- Duration: 10–15 minutes presentation + 5–10 minutes Q&A per team.
- Live demo required — slides alone are not sufficient. Have a backup recording in case of connectivity issues.
- All team members must speak and must be able to answer questions about any part of the system, including teammates' components.

Suggested Presentation Flow

21. Problem & motivation (1 slide)
22. Architecture overview: data → embeddings → vector store → RAG → interface → deployment (1–2 slides/diagram)
23. Live demo of the deployed application
24. Key technical decisions and trade-offs (prompting strategy, framework choice, distance metric, fine-tuning decision)
25. Limitations and what you would improve with more time
26. Open Q&A / defense

Sample Defense Questions

- Why did you choose this distance metric / vector database over the alternatives?
- Walk us through what happens, step by step, when a user submits a query.
- How does your system avoid or reduce hallucinated answers?
- What would break first if your document collection grew 100x?
- If you had time to fine-tune a model with LoRA/QLoRA, what would you target and why?

7. Repository & Documentation Requirements

Every checkpoint submission lives in the same GitHub/GitLab repository, which by the Final checkpoint should contain:

- README.md — setup instructions, architecture diagram/description, and how to run the app and the Docker container.
- /data — raw and processed datasets (or a link, if too large for the repo).
- /src or /app — all source code, organized by pipeline stage (preprocessing, embeddings, retrieval, interface).
- Dockerfile and any docker-compose/config files.
- /docs — your Project Proposal, Prompt Engineering Document, Framework Comparison Note, Model Customization Note, and Final Technical Report.
- Commit history showing incremental progress across all four checkpoints, not a single last-minute upload.

8. How This Project Fits Your Final Grade

Per the course grading scheme, Project work is 20% of your final grade. Within that 20%, your four checkpoints are weighted as follows:

Checkpoint	Performance Task	Exam Period	Weight of Project Grade
Checkpoint 1 – Foundation & Data Pipeline	PT 1	Prelim	20%
Checkpoint 2 – Prompts & Vector Indexing	PT 2	Midterm	20%
Checkpoint 3 – RAG Orchestration	PT 3	Semi-Final	30%
Checkpoint 4 – Deployment & Defense	PT 4	Final	30%
TOTAL			100%

Late or missing checkpoints carry forward: each checkpoint depends on artifacts from the one before it, so falling behind early compounds quickly. Talk to your instructor as soon as you are stuck — not at the deadline.